

Searching Binary Embeddings

Derivation and Experiment

12 September 2026

Abstract

Clustered scanning, bitplane search and weighted bit-pattern lookup are compared under common recall and memory limits. Formulas guide independent parameter choices. Scanning remained fastest on real embeddings. In constructed cases, backward walk was about 10% faster for fixed important coordinates; bitplanes were about 152 times faster when they changed. Revised predictions for six selected settings were within 10% of separate-query measurements.

1 Introduction

1.1 The shared pipeline

The baseline follows Exa’s description of Matryoshka prefixes, binary documents, float queries and clustered lookup-table scoring [1]. We change local candidate selection. These measurements concern this project’s implementations, not Exa’s database. Filters and reranking are optional shared stages.

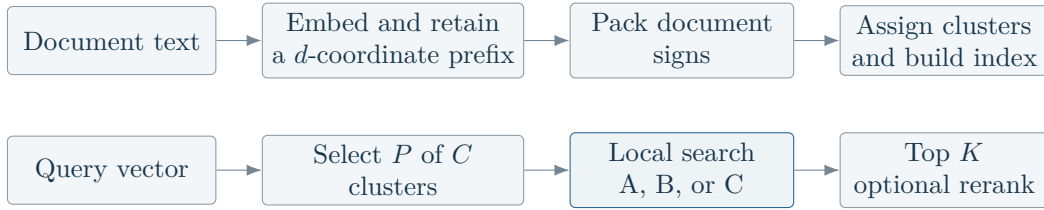


Figure 1: Blue marks the local stage under comparison. Embedding and index construction precede timed retrieval.

1.2 The query’s preferred pattern

A query reveals which binary pattern would maximize its score. For $q = [0.7, 0.5, -0.4, -0.6, 0.3]$, that pattern is 11001. A stored 1 represents +1, and 0 represents −1. Matching every sign gives $Q = \sum_i |q_i| = 2.5$; a mismatch at coordinate i loses $2|q_i|$. Therefore

$$S(q, b) = Q - 2p, \quad p = \sum_{i: b_i \neq \text{sign}(q_i)} |q_i|. \quad (1)$$

For example, 11000 has penalty 0.3 and score 1.9. The preferred pattern need not exist in the collection. The question is whether an index can locate sufficiently close patterns more cheaply than it can score every selected document.

1.3 Baseline: cluster and scan

Method A scores every selected document. Exa’s described lookup-table method [1] combines prepared contributions from small coordinate groups; Section 2 counts this work.

```

scan(query, selected_clusters, K):
    table, best = prepare_score_table(query), empty_top_k(K)
    for cluster in selected_clusters:
        for doc in cluster.documents:
            best.offer(full_score(table, doc), doc.id)
    return best.sorted_by_score_then_id()
  
```

1.4 Bitplanes

Method B keeps the packed rows and adds a second arrangement of their signs. Each *bitplane* stores one coordinate across all documents in a cluster. The figure shows three documents from the running example. A candidate mask records which local rows remain.

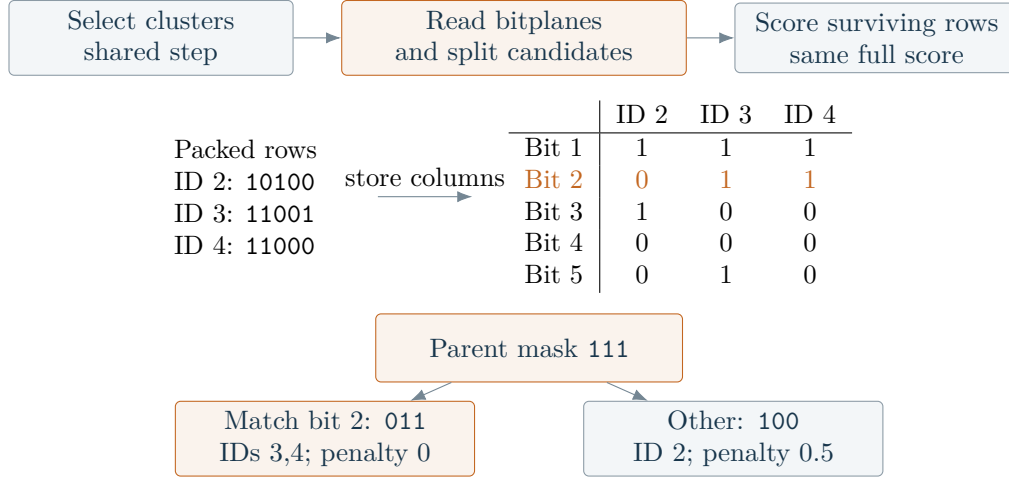


Figure 2: Orange marks the changed stage and the selected plane. Both child masks retain all three positions. On a larger cluster, this physical width is a major cost.

The query magnitudes order the coordinates: 1, 4, 2, 3, 5 in this example. For a positive query coordinate, a parent mask A is split into $A \& B_i$ and $A \& \neg B_i$. The first child matches; the other adds $|q_i|$ to its penalty. Negative query coordinates reverse the roles. Alternatives remain in a priority queue, ordered by known penalty, because a preferred early bit does not determine the final ranking.

Equation 1 gives a safe upper bound: a branch with known penalty p can score at most $Q - 2p$. If IDs 3 and 4 score 2.5 and 1.9, the alternative with penalty 0.5 has bound 1.5 and cannot improve the top two. Equal-score cases retain the common document-ID tie rule.

```
bitplanes(query, clusters, K, leaf_size, node_budget):
    table, best = prepare_score_table(query), empty_top_k(K)
    order = coordinates_by_decreasing_abs(query)
    queue = full candidate mask for each selected cluster
    while queue is not empty:
        node = remove_lowest_penalty(queue)
        if cannot_improve(node, query, best): continue
        if node_budget_is_reached(): break
        if node.count <= leaf_size or no_bits_remain(node):
            score_set_documents(table, node.mask, best)
        else:
            queue_nonempty_children(split(node, order))
    return best.sorted_by_score_then_id()
```

The leaf size decides when to stop splitting and score the remaining rows. Unlimited search with safe bounds is exact within opened clusters; a finite work limit can lose results.

1.5 Backward walk

Method C forms a short key from a fixed set of h coordinates. In each cluster, documents with the same key are stored consecutively. A directory maps an occupied key to the start and length of that row range. Search begins at the query's preferred key and enumerates alternatives in increasing weighted mismatch penalty.

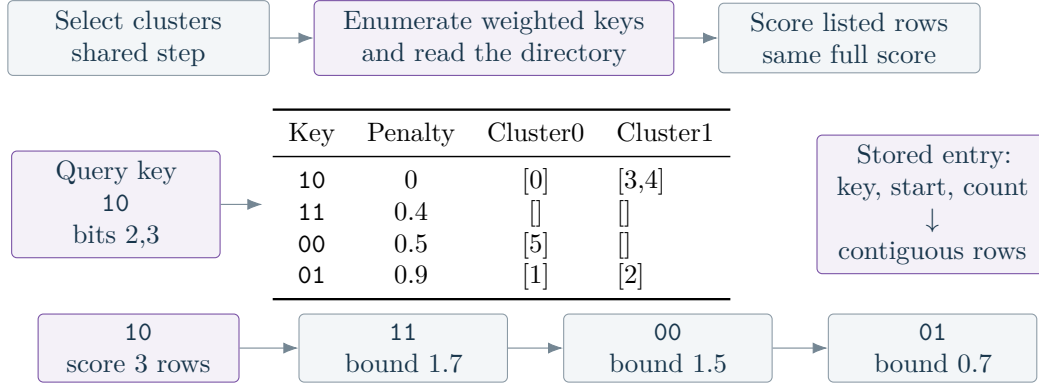


Figure 3: Purple marks the changed stage. Here the first key returns IDs 0,3,4. Their top two scores are 2.5 and 1.9, so the next key's bound 1.7 already permits stopping. The remaining entries show the order that a query requiring more work would follow.

The ordering uses the sum of magnitudes at flipped coordinates. Counting flipped bits alone would give the wrong order when query weights differ. There is one shared enumeration queue across opened clusters, followed by a lookup in each cluster's directory. Empty lookups also take time. Removing a key constraint is not a single ordinary hash lookup: the newly admitted patterns must be enumerated or supported by a different data structure.

```
backward_walk(query, clusters, K, key_coordinates, limits):
    table, best = prepare_score_table(query), empty_top_k(K)
    keys = unique_keys_by_increasing_penalty(query, key_coordinates)
    for key, penalty in keys:
        if cannot_improve(penalty, query, best): break
        for cluster in clusters:
            if work_limit_reached(limits):
                return best.sorted_by_score_then_id()
            rows = cluster.directory.lookup(key)
            score_each_row(table, rows, best)
    return best.sorted_by_score_then_id()
```

A short key omits some coordinates, so every retrieved row still receives the complete score. Finding enough candidates does not by itself prove sufficient recall. Exhausting the keys or ruling out every remaining bound is exact within opened clusters; stopping at a candidate or lookup budget is approximate. Neither B nor C can recover a document excluded by the initial routing.

2 Preliminary calculations

2.1 Time complexity

Let N be the document count, d the code length, C the cluster count, P the number of opened clusters, and K the requested results. A cluster c has n_c documents and $W_c = \lceil n_c/w \rceil$ words per dense mask, where $w = 64$ in the implementation. A complete document score uses $G = \lceil d/g \rceil$ table terms, with group width $g = 4$ in the measurements.

A: scan. The row count is $F_A = \sum_{c \in \mathcal{P}} n_c$, where $\mathcal{P}$ contains the selected clusters. The approximation NP/C requires balanced clusters. The scoring work is $F_A G$ table terms. Keeping a top- K heap has worst-case work $O(F_A \log K)$, although many scores do not change the retained set. Measured cost per row can include both.

B: bitplanes. Let J_c count splits and E_c count enumerated leaf masks. The word visits are $V_s = \sum_c J_c W_c$ and $V_l = \sum_c E_c W_c$. The number of scored documents is F_B . Sorting query coordinates costs $O(d \log d)$; the queue adds work for each visited branch. Counts of surviving candidates alone omit full-width split and leaf reads.

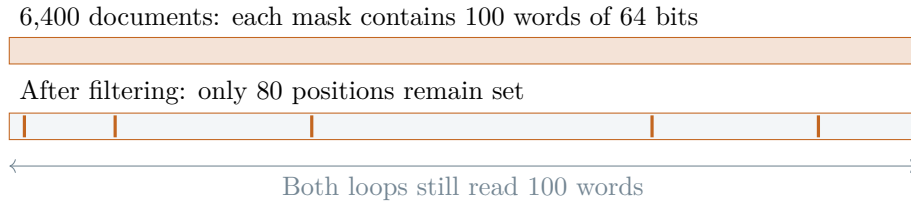


Figure 4: Logical membership decreases; physical mask width remains fixed. Marks in the lower bar illustrate surviving positions rather than drawing all 80 individual bits.

C: backward walk. Let E be enumerated keys and H all directory lookups. For complete enumeration across P clusters, $H = PE$. A hash lookup has expected constant algorithmic work, but includes memory access. Key generation, queue maintenance, empty lookups, and the F_C full scores remain separate costs. More key bits can reduce candidates while increasing enumeration work and stored directory entries.

Time summary

Write T_R for routing time and T_Q for the common score-table preparation. The measured coefficients $c_s, c_b, c_l, c_g, c_h, c_p$ price a consecutive score, split-word visit, leaf-word visit, gathered score, directory lookup, and posting score, respectively.

Method	Retrieval-time model
A	$T_R + T_Q + F_A c_s + T_{\text{select}}$
B	$T_R + T_Q + T_{\text{order}} + V_s c_b + V_l c_l + F_B c_g + T_{\text{queue}} + T_{\text{select}}$
C	$T_R + T_Q + T_{\text{keyprep}} + H c_h + F_C c_p + T_{\text{keyqueue}} + T_{\text{select}}$

These formulas explain work; coefficients convert work to time on a stated machine. SIMD, cache behavior and memory bandwidth change those coefficients. A combined measured stage must not be charged again as separate sub-stages. Complete text requests additionally include embedding, normalization, enabled filters, output work, and any reranker.

2.2 Memory complexity

Memory has two roles: the stored index and temporary allocations that coexist during a query. Let $u = w/8$ bytes per packed word and b_{id} bytes per document ID. The common row data occupies

$$M_{\text{rows}} = N (u \lceil d/w \rceil + b_{\text{id}}). \quad (2)$$

For 256 bits and eight-byte IDs, each row contains 32+8=40 bytes. Array capacity may exceed the number of filled rows. Router data is additional: for example, C float centroids of width d_R require $4Cd_R$ coordinate bytes, plus labels and supporting containers.

A: scan. A stores the rows and router. Query memory includes the float query, the score table of $G2^g$ values, selected cluster IDs, and top- K results. Their main size terms are $O(d + G2^g + P + K)$, with byte widths applied to each array.

B: bitplanes. B keeps the rows and adds

$$M_{\text{planes}} = du \sum_{c=1}^C \lceil n_c/w \rceil. \quad (3)$$

Each cluster is padded separately. With $A_c(t)$ allocated masks for cluster c at time t , peak mask data is $u \max_t \sum_c A_c(t) W_c$. Waiting branches and the parent mask during child creation must be counted, not just the currently selected mask. Queue records are additional.

C: backward walk. C reorders the existing rows and IDs. The IDs already serve as postings, so there is no additional copy of all N IDs. If U occupied directory entries store a key, start and count, their field data is $U(b_{\text{key}} + 2b_{\text{offset}})$ bytes. The current hash table also allocates nodes, pointers and a bucket array; those cannot be inferred from field sizes alone. A query also maintains its key queue.

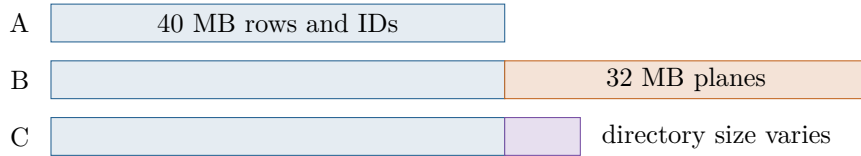


Figure 5: Logical data at 1M rows and 256 bits. The directory bar is schematic. Router, array capacity, program memory, and temporary query allocations are not included in these bars.

Memory summary

Let M_0 include allocated rows and routing structures, Q_0 common query allocations, D_C the allocated directory, and Q_B, Q_C the extra method-specific query memory.

Method	Allocated index	Temporary query memory
A	M_0	Q_0
B	$M_0 + M_{\text{planes,allocated}}$	$Q_0 + Q_B$
C	$M_0 + D_C$	$Q_0 + Q_C$

The actual RAM test also charges the running process and retained inputs. A process lifetime peak includes construction and provides a conservative upper bound for that measured run. It is distinct from index data size and from a sampled query-memory reading.

2.3 Recall estimation

Recall compares returned IDs with an explicitly chosen reference:

$$R_K(q) = \frac{|\text{returned top-}K \cap \text{reference top-}K|}{K}. \quad (4)$$

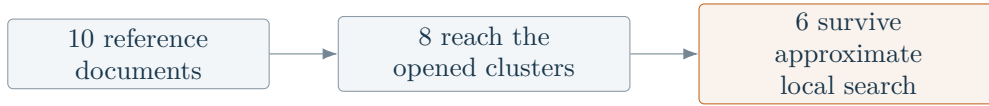
Returning IDs $[3,0]$ when the reference is $[3,4]$ gives 50% recall. Here the reference is the exact binary-document score. This differs from recovering the full float-vector ranking or retrieving text judged relevant by people.

For a fixed query, select a reference document uniformly from its top K . Let $R_R(q)$ be the probability its cluster is opened. Local recall is conditional on reaching that cluster.

A: scan. Exact scoring returns every reference document that reaches it. Its recall is $R_R(q)$.

B: bitplanes. Recall is $R_R(q)R_B^{\text{local}}(q)$. A node budget can reduce the local factor; safe unlimited search makes it one.

C: backward walk. Recall is $R_R(q)R_C^{\text{local}}(q)$. A lookup or candidate budget can reduce the local factor; exhaustion or safe bounds make it one.



$$R = (8/10)(6/8) = 60\%$$

Figure 6: The factors are conditional probabilities. Independence is not assumed. Across queries, average their products rather than multiply separate averages.

A distribution that permits a calculation

Assume independent balanced document signs. Each query has m strong coordinates of magnitude a and $d-m$ weak coordinates of magnitude ϵ , with $a > (d-m)\epsilon$. All documents matching the strong signs then outrank every document with a strong mismatch. Their count is $Z \sim \text{Binomial}(N, 2^{-m})$. If only that group is searched and fully scored,

$$\mathbb{E}[R_K] = \frac{\mathbb{E}[\min(Z, K)]}{K} = \frac{1}{K} \sum_{i=1}^K \Pr(Z \geq i). \quad (5)$$

For two documents, one required sign and $K = 1$, the ideal group is empty with probability $1/4$, so expected recall is $3/4$. Its mean size is one, but using that mean as a guaranteed count would incorrectly predict perfect recall. This distribution is constructed; Matryoshka training does not promise it or sort each query’s coordinates by magnitude [2].

Recall summary

Method	General per-query recall	Exact local search
A	$R_R(q)$	Scores all opened rows
B	$R_R(q)R_B^{\text{local}}(q)$	Unlimited work and safe bounds
C	$R_R(q)R_C^{\text{local}}(q)$	Exhaustion or safe bounds

Equation 5 estimates all three only when they search the specified ideal group. Arbitrary IVF routing and limited local searches require additional assumptions or measured tuning recall.

3 Deriving optima

3.1 The constrained choice

Let θ_a contain method a 's router, cluster count, probes and local settings. The question is

$$\theta_a^* = \arg \min_{\theta \in \Theta_a} T_a(\theta) \quad \text{subject to} \quad R_a(\theta) \geq \rho, \quad M_a(\theta) \leq B_{\max}. \quad (6)$$

The family Θ_a must allow each method its own cluster layout. B must also be allowed to stop splitting immediately; otherwise an unnecessary traversal is forced into its cost. A receives direct prefix routing as well as IVF. The desired minimum is meaningful only when the time, recall and memory functions are specified or estimated from evidence.

3.2 Cluster size and probes

For balanced clusters and a router that compares all centroids, a simple scan model is $T_A(C) = \alpha C + \beta NP/C$. Holding P fixed gives

$$\frac{dT_A}{dC} = \alpha - \frac{\beta NP}{C^2} = 0, \quad C^* = \sqrt{\frac{\beta NP}{\alpha}}. \quad (7)$$

This does not solve the fixed-recall problem when more clusters also require more probes. The relevant function is $P_\rho(C)$: the probes needed for recall ρ . Substituting it gives $T_A(C) = T_R(C, P_\rho(C)) + \beta NP_\rho(C)/C$. The experiment obtains this relationship from tuning-query reference ranks. It is not inferred from document count and RAM alone.

3.3 When to stop splitting

For one global cluster, suppose each useful cut halves the remaining documents and one preferred path contains the reference top K . At depth j ,

$$T_B(j) \approx T_0 + jWc_b + Wc_l + N2^{-j}c_g. \quad (8)$$

The first terms charge full-mask work; the last charges the remaining scores. Differentiating the smooth approximation gives

$$j^* = \log_2 \left(\frac{N \ln(2)c_g}{Wc_b} \right). \quad (9)$$

Depth must be an integer, so test the neighboring integers, zero and the largest depth for which the assumptions hold. Equivalently, one further cut helps when its saved score time $N2^{-(j+1)}c_g$ exceeds Wc_b and any added queue work. Since W grows with N , increasing the collection size does not automatically make branching favorable.

The implementation visits all coordinates in global query-magnitude order, including bits already fixed by a prefix cluster. With r such initial coordinates, $n = N/2^r$, and $W_r = \lceil n/w \rceil$, a path ending at absolute depth $j \geq r$ has approximately

$$T_B(j; r) = T_0 + jW_rc_b + W_rc_l + n2^{-(j-r)}c_g.$$

The corresponding stationary depth is $r + \log_2(n \ln(2)c_g / (W_rc_b))$. A zero-split leaf is a separate candidate. Subtracting the r fixed bits from measured split work would make the current implementation appear cheaper than it is.

The experiment retains the model's choice and the fastest measured tuning choice separately. A fitted time estimate cannot resolve a difference smaller than its observed error.

3.4 Fixed and changing coordinates

The controlled study uses $N = 10^6$, $d = 256$, $K = 100$, and $m = 12$ strong coordinates of magnitude 1, with magnitude 0.0001 elsewhere. The strong-match group has

$$\lambda = N2^{-m} = 244.14, \quad \sigma = \sqrt{N2^{-m}(1 - 2^{-m})} = 15.62.$$

A leaf near $\lambda + 3\sigma \approx 291$ suggests testing 288 and 320, rather than choosing an arbitrary threshold. The preceding group has mean 488 rows. Every query remains in the test, including groups outside this allowance; the statistical margin is not a size guarantee.

Fixed important positions. Suppose the first m coordinates are strong. With $r \leq m$ direct prefix bits and one probe, A scores about $N2^{-r}$ rows. At $r = m$, it reads the complete ideal group. Adding weak routing bits requires opening their compatible patterns, so it need not reduce the number of full scores.

C can divide the same m strong coordinates between r routing bits and $h = m - r$ local key bits. It still scores about $N2^{-m}$ rows. The expected number of occupied directory entries in this uniform case is

$$\mathbb{E}[U] = 2^{r+h} \left[1 - (1 - 2^{-(r+h)})^N \right]. \quad (10)$$

A direct 12-bit scan and a global 12-bit backward walk therefore select the same ideal group. Their difference is routing, directory organization and call overhead. This prevents an unfair comparison against a baseline that unnecessarily scans the entire collection.

Changing important positions. A fixed h -coordinate key captures a random number S of strong positions:

$$\Pr(S = s) = \frac{\binom{m}{s} \binom{d-m}{h-s}}{\binom{d}{h}}. \quad (11)$$

If all compatible weak patterns must be examined, the retained fraction is 2^{-S} and the attempted key count is 2^{h-S} . With $h = m = 12$, the key misses every strong coordinate in about 55.48% of queries. The expected retained fraction is about 75%, far above the ideal group's 0.0244%. This explains why the same key can be excellent for one distribution and expensive for another. The average row count does not by itself determine median latency.

B instead selects its coordinates from each query. A global path through twelve strong bits visits $12 \lceil 10^6/64 \rceil = 187,500$ split words, then 15,625 leaf words, and scores about 244 rows. Its conditional peak mask data is $(12 + 2)15,625 \times 8 = 1.75$ MB. Whether routing improves this path is tested separately, including its preparation and extra root-mask work.

Condition	Promising settings	What must be checked
Accurate, inexpensive routing	A with small selected groups	Recall at the selected probes
Fixed strong coordinates	A direct prefix; C global key near m	Same candidates; compare total overhead
Strong positions change	B with global or larger clusters	Full masks, leaf size, routing alternatives

These are predictions for testing. They do not require a different winner in every row, and they do not claim that real Matryoshka vectors satisfy the constructed distribution.

4 Experiments

4.1 Setup and parameter selection

The experiment uses one million 256-bit documents, top 100 results, a 32 GB decimal worker budget, one CPU thread, and one query at a time on an Apple M5 Max with 128 GiB of physical memory. A, B and C share the complete binary score and ID tie rule. Routing and the Python-to-C++ call are included in retrieval time; document preparation and query encoding are excluded. Fresh processes build and warm each configuration before measurement.

The controlled corpus retains the earlier random document signs after hash verification. Fresh queries use seed 20260912: 32 for tuning and 64 for evaluation in each of the fixed and changing-position cases. Both have twelve strong coordinates. Parameters are frozen before the evaluation; no query is removed because its match count or timing is inconvenient.

The declared family contains no-routing search, direct prefix widths 4,8,10,11,12,13,14, and IVF cluster counts 256 and 4096. Probe counts come from tuning-only recall cutoffs and the fixed-prefix calculation. Every method receives every available layout and probe count. B additionally varies leaf size, including 288 and 320 from the derivation and a scan-sized leaf. C varies key width and uses coordinates after the routing prefix. Both local searches have unlimited work budgets in this comparison.

There are 130 fixed-position and 299 changing-position tuning settings. Each has two timing repetitions. Each chosen setting is evaluated in three process blocks with two repetitions per query. Worker order is shuffled. Memory and power settings are recorded; unchanged settings do not prove constant CPU frequency. Process memory includes construction, which is stricter than checking only the stored index.

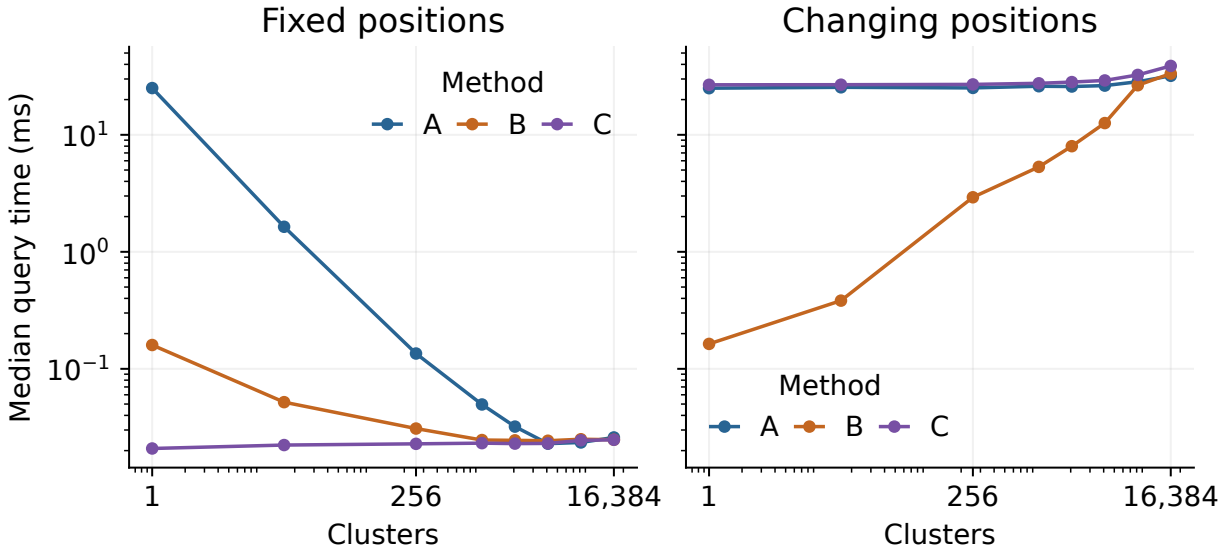


Figure 7: The fastest eligible *tuning* setting at each cluster count, after allowing each method its local settings and the available routers. These curves expose the cluster-size choice. They do not select new parameters from evaluation data or interpolate untested optima.

The measured selection uses each method’s fastest tuning setting whose lower recall estimate meets 99%. A separate selection minimizes the calibrated time model over eligible settings. Analytic work estimates are used only for supported uniform-sign paths; other cases use explicitly labeled tuning work estimates. Routing cost and recall eligibility remain empirical for arbitrary layouts. Both selections are evaluated when they differ.

4.2 Checking and revising the model

The first model correctly counted the applicable search work but assigned overly simple prices to it. Across tuning cases, 380 fixed-position and 253 changing-position count predictions matched native counters exactly. Other observations remained in the study, outside those particular formulas. However, some initial B/C time estimates differed by up to 69%.

One B example, a global index with leaf size 4096, was predicted at 0.265 ms but measured at 0.600 ms. A zero leaf word starts no scoring; a nonempty word begins a short run of row accesses. The first model did not distinguish this access pattern from scoring consecutive rows. Under uniformly distributed survivors, the expected nonempty leaf-word count is

$$V_{\text{nonempty}} \approx V_l \left[1 - \left(1 - \frac{F_B}{wV_l} \right)^w \right]. \quad (12)$$

For C, an empty directory lookup does less work than a successful lookup that starts a document range. A uniform-occupancy approximation estimates successful posting starts as

$$H_{\text{hit}} \approx H \frac{U}{C_{\text{stored}} 2^h}. \quad (13)$$

Here U is the occupied-entry count and C_{stored} the number of stored cluster directories. Query-selected cells may not be representative, so this remains an approximation.

The revised model adds a fitted cost for each of these terms; the failed first fit remains saved. Using the revised coefficients in the earlier depth guide gives approximately 13.5 for fixed positions and 11.5 for changing positions. Only depths 0–12 satisfy this experiment’s strong-coordinate assumption. The added nonempty-word term is nonlinear, so every valid integer depth is checked under the revised model. Both conditions select twelve cuts for the global path.

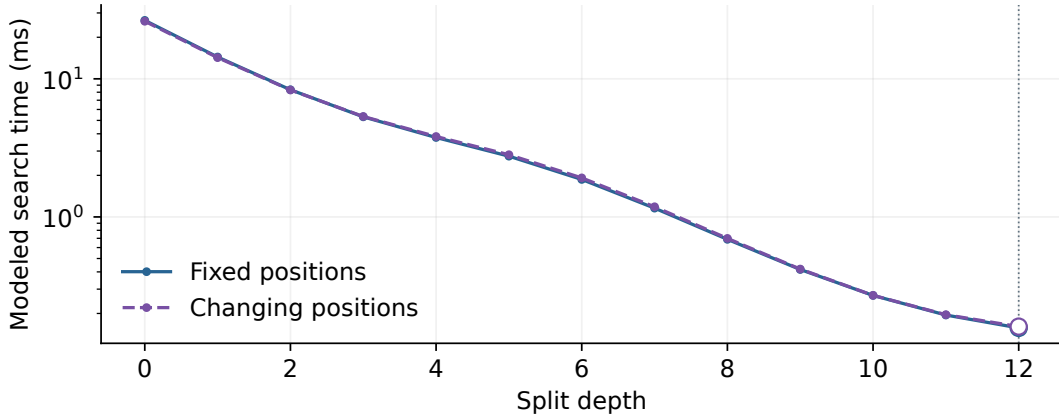


Figure 8: The revised global B model evaluated at all thirteen valid integer depths, including zero. The curve describes the constructed single-path case and calibrated costs. It does not assume that arbitrary real queries follow one branch.

The revision reduced the changing-position B maximum tuning error from 43.2% to 19.2%. Some C tuning errors still reach 39% in the changing case and 52% in the fixed case. The 20% acceptance threshold was not changed. These remaining failures limit the model’s range; a small predicted difference is insufficient to establish a winner. Final evaluation checks the selected settings without refitting their coefficients.

4.3 Controlled comparisons

Table 1: Frozen tuning choices on separate queries. Peak memory includes construction.

Condition	Method	Route C/P	L	h	Recall (%)	ms	Peak MB
Fixed	A	Prefix 4,096/1	–	–	100.00	0.0227	189.5
Fixed	B	Prefix 4,096/1	N	–	100.00	0.0234	225.4
Fixed	C	Global 1/1	–	12	100.00	0.0207	136.7
Changing	A	Global 1/1	–	–	100.00	25.0578	154.0
Changing	B	Global 1/1	288	–	100.00	0.1649	185.8
Changing	C	Global 1/1	–	8	100.00	26.7614	136.8

The settings in the table were selected using tuning queries, then measured on 64 separate queries. Their peak process memory is well below 32 GB. The fixed-position scan chooses 4096 direct prefix clusters and one probe. Backward walk chooses a global 12-bit key; both select the same ideal group. B’s selected large leaf performs no splits in this case.

When important coordinates change, the selected bitplane setting instead uses one global cluster and leaf size 288. Its query-dependent coordinate order removes most document scores without requiring a fixed key to capture the important positions. A and C also had access to all declared cluster layouts; neither was forced to use the same router as B.

Table 2: Paired time ratios. A ratio above one favors the method in the denominator.

Condition	Time ratio	Estimate	95% interval	Faster
Fixed	A/B	0.971	[0.932, 1.029]	–
Fixed	A/C	1.099	[1.070, 1.127]	C
Fixed	B/C	1.131	[1.094, 1.158]	C
Changing	A/B	151.980	[150.917, 152.810]	B
Changing	A/C	0.936	[0.906, 0.942]	A
Changing	B/C	0.00616	[0.00595, 0.00623]	B

Each interval resamples queries and whole process blocks with paired method observations. Repeated timings are not additional independent queries. A speed advantage is supported only when both choices meet the recall and memory requirements and the entire 95% ratio interval exceeds one. The fixed-position C advantage concerns the implemented routing and lookup organization; it does not establish an asymptotic advantage over direct prefix scanning.

Table 3: Frozen model choices: predicted and measured median retrieval times.

Condition	Method	Predicted ms	Measured ms	Error (%)	Recall (%)	Same setting
Fixed	A	0.0245	0.0223	+9.7	100.00	Yes
Fixed	B	0.0226	0.0240	-6.0	100.00	No
Fixed	C	0.0198	0.0210	-5.6	100.00	Yes
Changing	A	26.2454	26.3998	-0.6	100.00	Yes
Changing	B	0.1630	0.1661	-1.9	100.00	Yes
Changing	C	27.8237	26.7609	+4.0	100.00	No

All six model-selected settings reached 100% recall and their median-time predictions were within 10% of the new measurements. Two selected parameter records differ from the measured-tuning choice: fixed B uses leaf 320 instead of a scan-sized leaf, and changing C uses one key bit instead of eight. The former still performs no split on these opened clusters. This agreement is evidence for the selected operating points, not a guarantee that every setting in the larger grid is predicted within 20%.

4.4 Real-data replication

The real-data control repeats the earlier frozen choices on the same 200 MS MARCO/Nomic evaluation queries [3, 4]. It is a replication, not a new-query generalization test. The original tuning covered 6821 settings; this run preserves each method’s chosen layout and local settings, then repeats them in three new process blocks.

Table 4: Replication of the frozen real-data choices at the 99% target.

Condition	Method	Route C/P	L	h	Recall (%)	ms	Peak MB
Real data	A	IVF 4,096/1,536	–	–	99.25	10.1050	183.4
Real data	B	IVF 4,096/1,536	1024	–	99.25	10.4267	220.5
Real data	C	IVF 4,096/1,536	–	4	99.25	12.5883	154.9

Table 5: Mean work per query. Timing repetitions are not extra count samples.

Condition	Method	Rows scored	Split words	Leaf words	Key lookups
Fixed	A	243.3	0.0	0.0	0.0
Fixed	B	243.3	0.0	4.2	0.0
Fixed	C	243.3	0.0	0.0	1.0
Changing	A	1,000,000.0	0.0	0.0	0.0
Changing	B	244.7	187,744.1	15,869.1	0.0
Changing	C	808,626.4	0.0	0.0	207.0
Real data	A	375,897.1	0.0	0.0	0.0
Real data	B	375,897.1	0.0	6,622.3	0.0
Real data	C	375,897.1	0.0	0.0	24,576.0

A was fastest in this replication. The paired 95% intervals for A/B and A/C were 0.963–0.974 and 0.787–0.818, respectively; both exclude one.

The work counts explain why an algorithm’s name is insufficient to interpret its timing. In the real 99% comparison, all three methods score the same selected documents. B’s large leaves avoid splitting but still require mask enumeration. C enumerates the four-bit keys inside the selected clusters. Neither alternative saves full document scores at that operating point, so their extra work cannot be interpreted as successful pruning.

The controlled case asks a different question. With changing important coordinates, B’s full-width mask work is worthwhile because it avoids nearly all row scores. With fixed important coordinates, A and C both obtain a small group directly. The parameter choices, native counters and complete retrieval timings must be read together.

4.5 Code and evidence

Paper component	Repository entry point
Three search implementations	<code>cpp/index.cpp</code> , <code>cpp/key_index.cpp</code> , <code>cpp/score.hpp</code>
Distribution and depth formulas	<code>experiments/optima_model.py</code>
Preparation, tuning, evaluation	<code>experiments/optima_study.py</code>
Initial/revised time predictions	<code>experiments/optima_predictions.py</code>
Native conditional count checks	<code>experiments/check_controlled.py</code>
All inputs, failures and observations	<code>results/optima-2026-09-12/</code>

The source README gives the exact commands. Raw cases are archived with file hashes; the first failed model remains in `predictions-v1/`. The plots and tables are generated from saved results. Rebuilding the PDF does not rerun the experiment.

5 Limitations and next steps

The optimized choices apply to the declared family and tested workload. The controlled distribution creates a clear score gap: twelve coordinates dominate all others combined. That is a useful test of the indexing idea, not evidence that typical text embeddings have the same structure. Matryoshka prefixes alone do not establish the assumption.

The cost model is more reliable for the selected operating points than across every setting. Memory access order, result-selection behavior and different query mixtures can change latency even when aggregate work counts match. The new occupancy terms improve the estimate without making it a universal hardware model. The next relevant check on real embeddings is whether a small, query-dependent coordinate set actually determines their nearest neighbors.

A larger collection does not automatically create a crossover. Under the simple fixed-depth model, write $T_A = t_A + fNc_s$ and $T_B = t_B + N\beta_B$, where f is A’s scanned fraction and $\beta_B = (mc_b + c_l)/w + 2^{-m}c_g$. Their candidate crossing is

$$N^* = \frac{t_B - t_A}{fc_s - \beta_B}.$$

It is useful only within the model’s valid range and while recall and RAM constraints hold. If B has both a higher fixed cost and a larger per-document cost, increasing N creates no win in this model. At one billion 256-bit documents, codes and eight-byte IDs alone need 40 GB; B adds at least 32 GB of bitplanes. All current layouts therefore exceed a 32 GB budget at that size, before program and temporary memory. No billion-document experiment is claimed.

6 Conclusion

Scanning was fastest in the real-data replication. In the constructed fixed-position case, backward walk was about 10% faster than a direct-prefix scan that selected the same candidate group. With changing important coordinates, bitplanes were about 152 times faster than the best tested scan. These results support different choices for different workloads; they do not establish a universal winner or the prevalence of those conditions in real embeddings.

The formulas identify candidates and explain their work. The final decision still combines measured recall, memory and complete retrieval time, with every method allowed its own settings. The selected-point prediction check, the retained failures and the real-data replication define the scope of that conclusion.

References

- [1] Exa Team. *How we built a web-scale vector database*. 2024. <https://exa.ai/blog/building-web-scale-vector-db>.
- [2] A. Kusupati et al. *Matryoshka Representation Learning*. 2022. <https://arxiv.org/abs/2205.13147>.
- [3] Microsoft. *MS MARCO*. Passage collection and benchmark resources. <https://microsoft.github.io/msmarco/>.
- [4] Nomic AI. *nomic-embed-text-v1.5*. Model documentation. <https://huggingface.co/nomic-ai/nomic-embed-text-v1.5>.